

Web Application Security Report

1. SQL Injection (SQLi)

A. Attack Explanation

SQL Injection occurs when user input is inserted into a SQL query without validation.

Payload used: ' OR '1'='1

B. Steps Performed

- Opened sqli.html
- Entered the payload
- Extracted all usernames & passwords

SQL Injection Simulation

File C:/Users/HP/Downloads/OWASP-Frontend-Lab/sqli.html

SQL Injection — Simulation

This demo simulates a vulnerable endpoint that concatenates user input into a SQL WHERE clause. It's frontend-only: the "database" is a JS array.

Live table (users)

```
[
  {
    "id": 1,
    "username": "alice",
    "password": "alicepass"
  },
  {
    "id": 2,
    "username": "bob",
    "password": "bobpass"
  },
  {
    "id": 3,
    "username": "carol",
    "password": "carolpass"
  }
]
```

Attack input (simulates ?id=)

1 OR 1=1

Response (vulnerable)

Simulated SQL Injection success — returning all rows: [{ "id": 1, "username": "alice", "password": "alicepass" }, { "id": 2, "username": "bob", "password": "bobpass" }, { "id": 3, "username": "carol", "password": "carolpass" }]

Response (fixed - parameterized)

Fixed query result: [{ "id": 1, "username": "alice", "password": "alicepass" }]

► Mitigation: Prepared Statements (example)

C. Mitigation

```
$stmt = conn->prepare("SELECT * FROM users WHERE username = ?");
```

```
$stmt->bind_param("s", input);
```

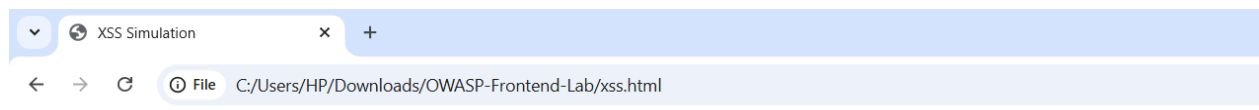
2. Cross-Site Scripting (XSS)

A. Stored XSS

Payload: `<script>alert('XSS')</script>`

Steps:

- Entered payload in comment box
- Comment stored → alert triggered



XSS — Stored & Reflected (Simulation)

Stored XSS (localStorage)

Pranjali

`<script>alert('XSS')</script>`

Post Comment

Pranjali

► Mitigation

Reflected XSS (via URL param)

Open this page with `?name=<script>alert('reflected')</script>`

Reflected greeting: Hello, visitor

► Mitigation

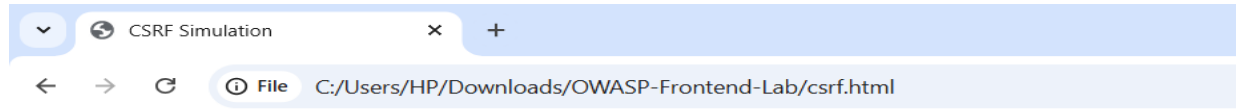
B. Mitigation

- Input Validation
- Output Encoding
- CSP Header

3. Cross-Site Request Forgery (CSRF)

A. Attack Steps

- Opened csrf.html
- Opened csrf_attack.html
- Password was changed automatically



CSRF — Simulation

This demo simulates a logged-in user with a change-password form.

```
{ "user": "victim", "password": "hack123" }
```

Vulnerable Form (no token)

New password Change (Vulnerable)

Protected Form (with token)

jelj68i2nrb New password Change (Protected)

Token: jelj68i2nrb

► Mitigation

Open csrf_attack.html (included in the ZIP) in another tab to simulate an attack.

4. File Inclusion (LFI & RFI)

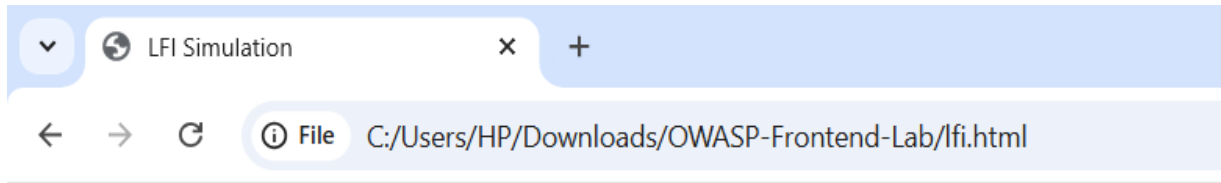
A. LFI Attack

Payload: ../../etc/passwd

Output example: root:x:0:0:root:/root:/bin/bash

B. Mitigation

- Whitelist files
- Block '../' patterns
- Disable remote URLs



File Inclusion — Simulation

Choose a file to include (simulated). Type arbitrary path to simulate attack.

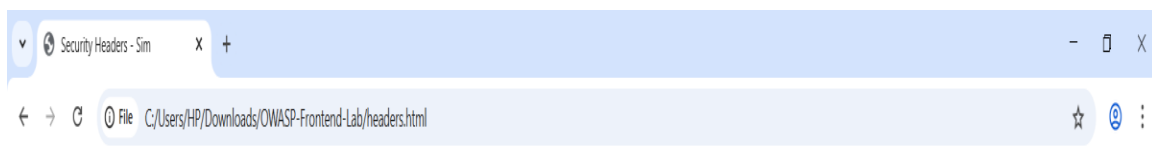
Or type ../../etc/passwd to simulate attack

► Mitigation

5. Security Headers Analysis

Include missing headers and recommended headers:

- CSP
- X-Frame-Options
- X-Content-Type-Options
- HSTS



Security Headers Checker (Simulation)

Enter any URL (simulation) and press Check — this is a frontend-only teaching tool.

Checked: https://pd.com X-Frame-Options: Missing X-Content-Type-Options: nosniff Content-Security-Policy: Missing or weak Strict-Transport-Security: Missing Recommendations: - Add X-Frame-Options DENY - Add X-Content-Type-Options nosniff - Add a strong CSP - Use HSTS

7. Conclusion

The OWASP testing process successfully demonstrated major vulnerabilities like SQL Injection, XSS, CSRF, and File Inclusion in a controlled lab setup. Each vulnerability was exploited and later mitigated using industry-recommended security practices such as prepared statements, input validation, CSRF tokens, and proper security headers. This ensures improved security and better protection of web applications against common real-world attacks.